

4.4 Histogram manipulation

What is the histogram of an image?

The histogram of an image is a discrete function that is formed by counting the number of pixels in the image that have a certain grey value. When this function is normalised to sum up to 1 for all the grey values, it can be treated as a probability density function that expresses how probable it is for a certain grey value to be found in the image. Seen this way, the grey value of a pixel becomes a random variable which takes values according to the outcome of an underlying random experiment.

When is it necessary to modify the histogram of an image?

If we cannot see much detail in an image, the reason could be that pixels, which represent different objects or parts of objects, have grey values which are very similar to each other. This is demonstrated with the example histograms shown in figure 4.37. The histogram of the “bad” image is very narrow and it does not occupy the full range of possible grey values, while the histogram of the “good” image is more spread. In order to improve the “bad” image, we might like to modify its histogram so that it looks like that of the “good” image.

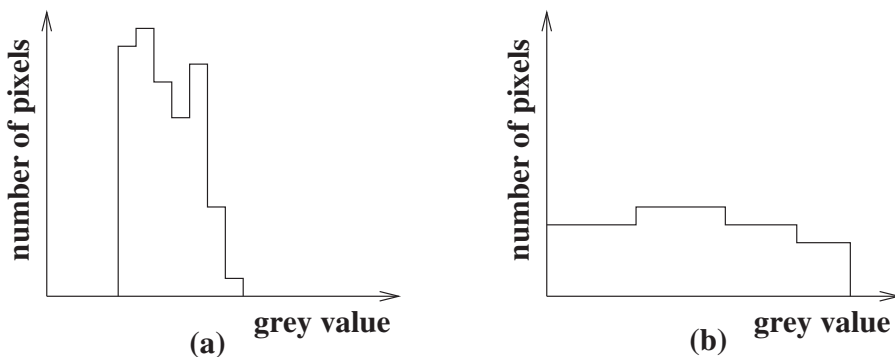


Figure 4.37: (a) The histogram of a “bad” image. (b) The histogram of a “good” image.

How can we modify the histogram of an image?

The simplest way is **histogram stretching**. Let us say that the histogram of the low contrast image ranges from grey value g_{min} to g_{max} . We wish to spread these values over the range $[0, G - 1]$, where $G - 1 > g_{max} - g_{min}$. We may map the grey values to the new range, if the grey value of a pixel g_{old} is replaced with the value g_{new} , given by:

$$g_{new} = \left\lfloor \frac{g_{old} - g_{min}}{g_{max} - g_{min}} G + 0.5 \right\rfloor \quad (4.133)$$

Term 0.5 was added so that the real number $(g_{old} - g_{min})G/(g_{max} - g_{min})$ is rounded by the floor operator $\lfloor \cdot \rfloor$ to its nearest integer as opposed to its integer part. We saw a version of this method on page 354, where equation (4.124) is used instead, designed to trim out extreme values.

Note that all we do by applying equation (4.133) is to spread the grey values, without changing the number of pixels per grey level. There are more sophisticated methods, which as well as stretching the range of grey values, allocate a predefined number of pixels at each grey level. These methods are collectively known as **histogram manipulation**.

What is histogram manipulation?

Histogram manipulation is the change of the grey values of an image, without affecting its semantic information content.

What affects the semantic information content of an image?

The information content of an image is conveyed by the *relative* grey values of its pixels. Usually, the grey values of the pixels do not have meaning in absolute terms, but only in relative terms. If the order (ranking) of pixels in terms of their grey value is destroyed, the information content of the image will be affected. So, an image enhancing method should preserve the relative brightness of pixels.

How can we perform histogram manipulation and at the same time preserve the information content of the image?

Let us assume that the grey values in the original image are represented by variable r and in the new image by variable s . We would like to find a transformation $s = T(r)$ such that the probability density function $p_{old}(r)$, which might look like the one in figure 4.37a, is transformed into a probability density function $p_{new}(s)$, which might look like that in figure 4.37b.

In order to preserve the information content of the image, all pixels that were darker than a pixel with grey value R , say, should remain darker than this pixel even after the transformation, when this pixel gets a new value S , say. So, for *every* grey value R , the number of pixels with lower grey values should be the same as the number of pixels with lower grey values than S , where S is the value to which R is mapped. This may be expressed by saying that the transformation T between the two histograms must preserve the distribution function of the normalised histograms:

$$\begin{aligned} P_{old}(R) &= P_{new}(S) \\ \Leftrightarrow \int_0^R p_{old}(r)dr &= \int_0^S p_{new}(s)ds \end{aligned} \quad (4.134)$$

This equation can be used to define the transformation T that must be applied to the value R of variable r to obtain the corresponding value S of variable s , provided we define function $p_{new}(s)$.

Example 4.23

The histogram of an image may be approximated by the probability density function

$$p_{old}(r) = Ae^{-r} \quad (4.135)$$

where r is the grey level variable taking values between 0 and b , and A is a normalising factor. Calculate the transformation $s = T(r)$, where s is the grey level value in the transformed image, such that the transformed image has probability density function

$$p_{new}(s) = Bse^{-s^2} \quad (4.136)$$

where s takes values between 0 and b , and B is some normalising factor.

Transformation $S = T(R)$ may be calculated using equation (4.134):

$$B \int_0^S se^{-s^2} ds = A \int_0^R e^{-r} dr \quad (4.137)$$

The left-hand side of (4.137) is:

$$\int_0^S se^{-s^2} ds = \frac{1}{2} \int_0^S e^{-s^2} ds^2 = -\frac{1}{2} e^{-s^2} \Big|_0^S = \frac{1 - e^{-S^2}}{2} \quad (4.138)$$

The right-hand side of (4.137) is:

$$\int_0^R e^{-r} dr = -e^{-r} \Big|_0^R = 1 - e^{-R} \quad (4.139)$$

We substitute from (4.138) and (4.139) into (4.137) to obtain:

$$\begin{aligned} \frac{1 - e^{-S^2}}{2} &= \frac{A}{B} (1 - e^{-R}) \Rightarrow \\ e^{-S^2} &= 1 - \frac{2A}{B} (1 - e^{-R}) \Rightarrow \\ -S^2 &= \ln \left[1 - \frac{2A}{B} (1 - e^{-R}) \right] \Rightarrow \\ S &= \sqrt{-\ln \left[1 - \frac{2A}{B} (1 - e^{-R}) \right]} \end{aligned} \quad (4.140)$$

So, each grey value R of the original image should be transformed into grey value S in the enhanced image, according to equation (4.140).

What is histogram equalisation?

Histogram equalisation is the process by which we make all grey values in an image equally probable, ie we set $p_{new}(s) = c$, where c is a constant. Transformation $S = T(R)$ may be calculated from equation (4.134) by substitution of $p_{new}(s)$ and integration. Figures 4.38a-4.38d show an example of applying this transformation to a low contrast image. Notice how narrow the histogram 4.38b of the original image 4.38a is. After histogram equalisation, the histogram in 4.38d is much more spread, but contrary to our expectations, it is not flat, ie it does not look “equalised”.

Why do histogram equalisation programs usually not produce images with flat histograms?

In the above analysis, we tacitly assumed that variables r and s can take continuous values. In reality, of course, the grey level values are discrete. In the continuous domain there is an infinite number of values in any interval $[r, r + dr]$. In digital images, we have only a finite number of pixels in each range. As the range is stretched, and the number of pixels in it is preserved, there is only a finite number of pixels with which the stretched range is populated. The histogram that results is spread over the whole range of grey values, but it is far from flat.

How do we perform histogram equalisation in practice?

In practice, r takes discrete values g , ranging between, say, g_{min} and g_{max} . Also, s takes discrete values t , ranging from 0 to $G - 1$, where typically $G = 256$. Then equation (4.134) becomes:

$$\sum_{t=0}^S p_{new}(t) = \sum_{g=g_{min}}^R p_{old}(g) \quad (4.141)$$

For histogram equalisation, $p_{new}(t) = 1/G$, so that the values of $p_{new}(t)$ over the range $[0, G - 1]$ sum up to 1. Then:

$$\frac{1}{G}(S + 1) = \sum_{g=g_{min}}^R p_{old}(g) \Rightarrow S = G \sum_{g=g_{min}}^R p_{old}(g) - 1 \quad (4.142)$$

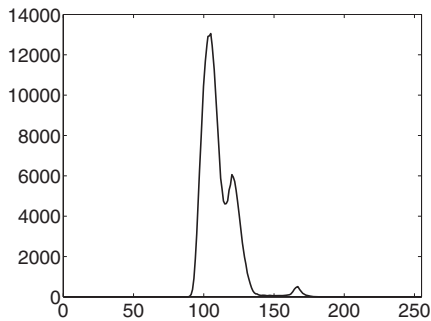
For every grey value R , this equation produces a corresponding value S . In general, this S will not be integer, so in order to get an integer value, we round it to an integer by taking its ceiling. This is because, when $R = g_{min}$, the first term on the right-hand side of equation (4.142) may be less than 1 and so S may become negative instead of 0. When $R = g_{max}$, the sum on the right-hand side of (4.142) is 1 and so S becomes $G - 1$, as it should be. So, finally a pixel with grey value g_{old} in the original image should get value g_{new} in the enhanced image, given by

$$g_{new} = \left\lceil G \sum_{g=g_{min}}^{g_{old}} p_{old}(g) - 1 \right\rceil \quad (4.143)$$

where $p_{old}(g)$ is the normalised histogram of the old image.



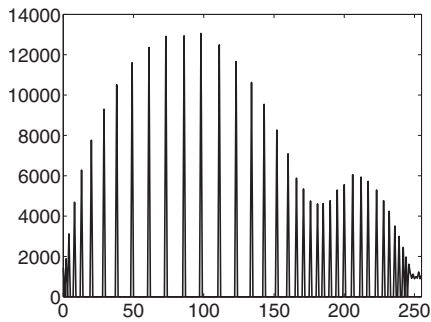
(a) Original image



(b) Histogram of (a)



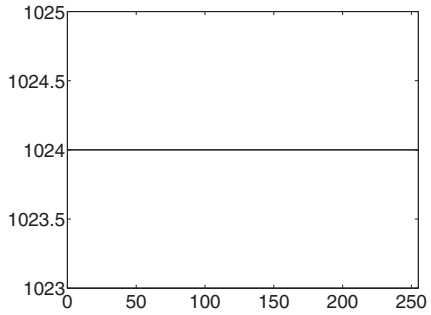
(c) After histogram equalisation



(d) Histogram of (c)



(e) After histogram equalisation with random additions

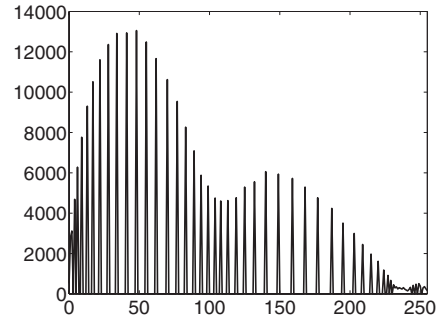


(f) Histogram of (e)

Figure 4.38: Enhancing the image of “The Bathtub Cleaner” by histogram equalisation.



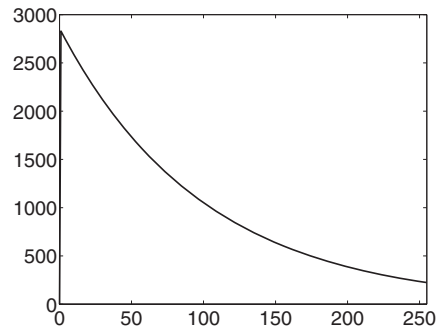
(a) After histogram hyperbolisation



(b) Histogram of (a)



(c) After histogram hyperbolisation with random additions



(d) Histogram of (c)

Figure 4.39: Histogram hyperbolisation with $\alpha = 0.01$ applied to the image of figure 4.38a.

Can we obtain an image with a perfectly flat histogram?

Yes, if we remove the constraint that the ranking of pixels in terms of their grey values has to be strictly preserved. We may allow, for example, pixels to be moved into neighbouring bins in the histogram, so that all bins have equal number of pixels. This method is known as **histogram equalisation with random additions**. Let us say that the (unnormalised) histogram of the image, after stretching or equalising it, is represented by the 1D array $H(g)$, where $g \in [0, G - 1]$, and that the image has NM pixels. The algorithm of histogram equalisation with random additions should work as follows.

Step 1: To the grey value of each pixel, add a random number drawn from a uniform distribution $[-0.5, 0.5]$.

Step 2: Order the grey values, keeping track which grey value corresponds to which pixel.

Step 3: Change the first $\lfloor \frac{NM}{G} \rfloor$ grey values to 0. Change the next $\lfloor \frac{NM}{G} \rfloor$ grey values to 1, ... etc until the last $\lfloor \frac{NM}{G} \rfloor$ which change to $G - 1$.

The result of applying this algorithm to the image of figure 4.38a is shown in 4.38e.

What if we do not wish to have an image with a flat histogram?

We may define $p_{new}(s)$ in (4.134) to be any function we wish. Once $p_{new}(s)$ is known (the desired histogram), one can solve the integral on the right-hand side to derive a function f_1 of S . Similarly, the integral on the left-hand side may be performed to yield a function f_2 of R , ie

$$f_1(S) = f_2(R) \Rightarrow S = f_1^{-1}f_2(R) \quad (4.144)$$

A special case of this approach is **histogram hyperbolisation**, where $p_{new}(s) = Ae^{-\alpha s}$ with A and α being some positive constants. The effect of this choice is to give more emphasis to low grey values and less to the high ones. This algorithm may also be used in conjunction with random additions, to yield an image with a perfectly hyperbolic histogram (see figure 4.39). In figure 4.39d this can be seen clearly because the method of random additions was used.

How do we do histogram hyperbolisation in practice?

Set $p_{new}(s) = Ae^{-\alpha s}$ in (4.134). First, we work out the value of A , noticing that $p(s)$ has to integrate to 1, from 0 to $G - 1$:

$$\begin{aligned} \int_{s=0}^{G-1} Ae^{-\alpha s} ds = 1 &\Rightarrow A \int_0^{G-1} e^{-\alpha s} d(\alpha s) \frac{1}{\alpha} = 1 \Rightarrow \frac{A}{\alpha} \int_0^{\alpha(G-1)} e^{-t} dt = 1 \Rightarrow \\ -\frac{A}{\alpha} e^{-\alpha(G-1)} + \frac{A}{\alpha} &= 1 \Rightarrow \frac{A}{\alpha} [1 - e^{-\alpha(G-1)}] = 1 \Rightarrow A = \frac{\alpha}{1 - e^{-\alpha(G-1)}} \end{aligned} \quad (4.145)$$

The right-hand side then of (4.134) becomes:

$$\int_0^S Ae^{-\alpha s} ds = A \frac{e^{-\alpha s}}{-\alpha} \Big|_0^S = \frac{A}{\alpha} (1 - e^{-\alpha S}) \quad (4.146)$$

For the discrete normalised histogram of the original image $p_r(g)$, equation (4.144) then takes the form:

$$\frac{A}{\alpha} (1 - e^{-\alpha S}) = \sum_{g=g_{min}}^R p_{old}(g) \Rightarrow S = -\frac{1}{\alpha} \ln \left[1 - \frac{\alpha}{A} \sum_{g=g_{min}}^R p_{old}(g) \right] \quad (4.147)$$

Since S has to take integer values, we add 0.5 and take the floor, so we finally arrive at the transformation:

$$g_{new} = \left\lfloor -\frac{1}{\alpha} \ln \left[1 - \frac{\alpha}{A} \sum_{g=g_{min}}^{g_{old}} p_{old}(g) \right] + 0.5 \right\rfloor \quad (4.148)$$

In figure 4.39, $\alpha = 0.01$ as this gave the most aesthetically pleasing result. It was found with experimentation that $\alpha = 0.05$ and $\alpha = 0.1$ gave images that were too dark, whereas $\alpha = 0.001$ gave an image that was too bright. For this particular image, $G = 256$.

How do we do histogram hyperbolisation with random additions?

The only difference with histogram equalisation with random additions is that now each bin of the desired histogram has to have a different number of pixels. First we have to decide the number of pixels per bin. If t denotes the discrete grey values of the enhanced image, bin $H(t)$ of the desired histogram will have $V(t)$ pixels. So, first we calculate the number of pixels we require per bin. This may be obtained by multiplying the total number of pixels with the integral of the desired probability density function over the width of the bin, ie the integral from t to $t + 1$. For an $N \times M$ image, the total number of pixels is NM . We then have

$$\begin{aligned} V(t) &= NMA \int_t^{t+1} e^{-\alpha t} dt \Rightarrow V(t) = NMA \left. \frac{e^{-\alpha t}}{-\alpha} \right|_t^{t+1} \\ \Rightarrow V(t) &= -NM \frac{A}{\alpha} [e^{-\alpha(t+1)} - e^{-\alpha t}] \Rightarrow V(t) = NM \frac{A}{\alpha} [e^{-\alpha t} - e^{-\alpha(t+1)}] \quad (4.149) \end{aligned}$$

where A is given by (4.145).

The algorithm then of histogram hyperbolisation with random additions is as follows.

Step 1: To the grey value of each pixel add a random number drawn from a uniform distribution $[-0.5, 0.5]$.

Step 2: Order the grey values, keeping track which grey value corresponds to which pixel.

Step 3: Set the first $\lfloor V(0) \rfloor$ pixels to 0.

Step 4: For t from 1 to $G - 1$, assign to the next $\lfloor V(t) + \{V(t - 1) - \lfloor V(t - 1) \rfloor\} \rfloor$ pixels grey value t . Note the correction term $V(t - 1) - \lfloor V(t - 1) \rfloor$ we incorporate in order to account for the left-over part of $V(t - 1)$, which, when added to $V(t)$ may produce a value incremented by 1 when the floor operator is applied.

Why should one wish to perform something other than histogram equalisation?

One may wish to emphasise certain grey values more than others, in order to compensate for a certain effect; for example, to compensate for the way the human eye responds to the different degrees of brightness. This is a reason for doing histogram hyperbolisation: it produces a more pleasing picture.

The human eye can discriminate better darker shades than brighter ones. This is known from psychophysical experiments which have shown that the threshold difference in brightness ΔI , for which the human eye can separate two regions, over the average brightness I is constant and roughly equal to 0.02 (see equation (4.129) of Box 4.11, on page 360). So, the brighter the scene, (higher I) the more different two brightness levels have to be in order for us to be able to discriminate them. In other words, the eye shows more sensitivity to dark shades and this is why histogram hyperbolisation is believed to produce better enhanced images, as it places more pixels in the dark end of the grey spectrum.



(a) Original image



(b) After global histogram equalisation

(c) Local histogram equalisation 81×81 (d) Local histogram equalisation 241×241 Figure 4.40: Enhancing the image of “A Young Train Driver” (of size 512×512).

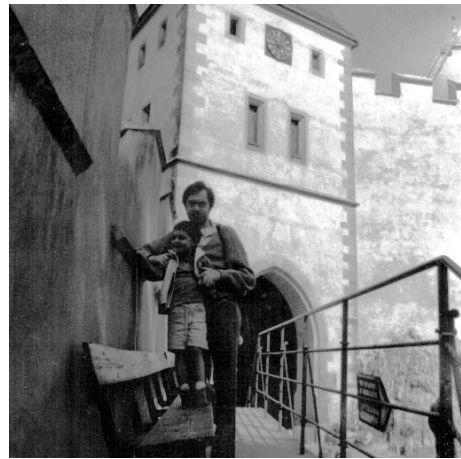
What if the image has inhomogeneous contrast?

The approach described above is global, ie we modify the histogram which refers to the whole image. However, the image may have variable quality at various parts. In that case, we may apply the above techniques locally: we scan the image with a window inside which we modify the histogram but we alter only the value of the central pixel. Clearly, such a method is costly and various algorithms have been devised to make it more efficient.

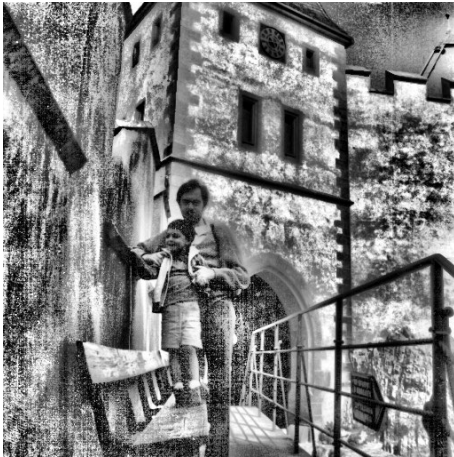
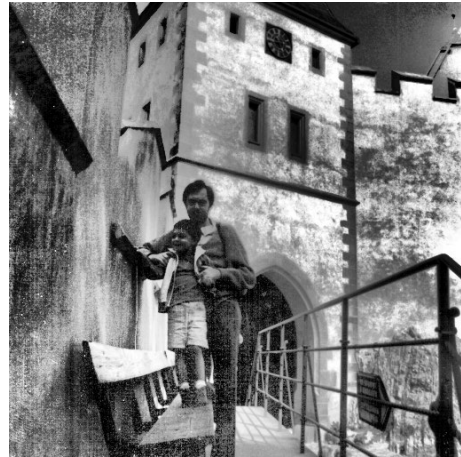
Figure 4.40a shows a classical example of an image that requires local enhancement. The picture was taken indoors looking towards windows with plenty of ambient light coming through. All outdoor sections are fine, but in the indoor part the film was under-exposed. The result of global histogram equalisation, shown in figure 4.40b, makes the outdoor parts over-exposed in order to allow us to see the details of the interior. The results of local histogram equalisation, shown in figures 4.40c and 4.40d, are overall much more pleasing.



(a) Original image



(b) After global histogram equalisation

(c) Local histogram equalisation 81×81 (d) Local histogram equalisation 241×241 Figure 4.41: Enhancing the image “At the Karlstejn Castle” (of size 512×512).

The window size used for 4.40c was 81×81 , while for 4.40d it was 241×241 , with the original image being of size 512×512 . Notice that no part of the image gives the impression of being over-exposed or under-exposed. There are parts of the image, however, that look damaged, particularly at the bottom of the image. They correspond to parts of the original film which received too little light to record anything. They correspond to flat black patches, and, by trying to enhance them, we simply enhance the film grain or the instrument noise.

A totally different effect becomes evident in figure 4.41c which shows the local histogram enhancement of a picture taken at Karlstejn castle in the Czech Republic, shown in figure 4.41a. The castle at the back consists of flat grey walls. The process of local histogram equalisation amplifies every small variation of the wall to such a degree that the wall looks like the rough surface of a rock. Further, on the left of the image, we observe again the effect of trying to enhance a totally black area. However, increasing the window size to 241×241 removes most of the undesirable effects.

Can we avoid damaging flat surfaces while increasing the contrast of genuine transitions in brightness?

Yes, there are algorithms that try to do that by taking into consideration pairs of pixel values. Such algorithms may be best understood if we consider the mapping function between input and output grey values. This function has to be one-to-one, but it may be chosen so that it stretches differently different ranges of grey values. This idea is shown schematically in figure 4.42.

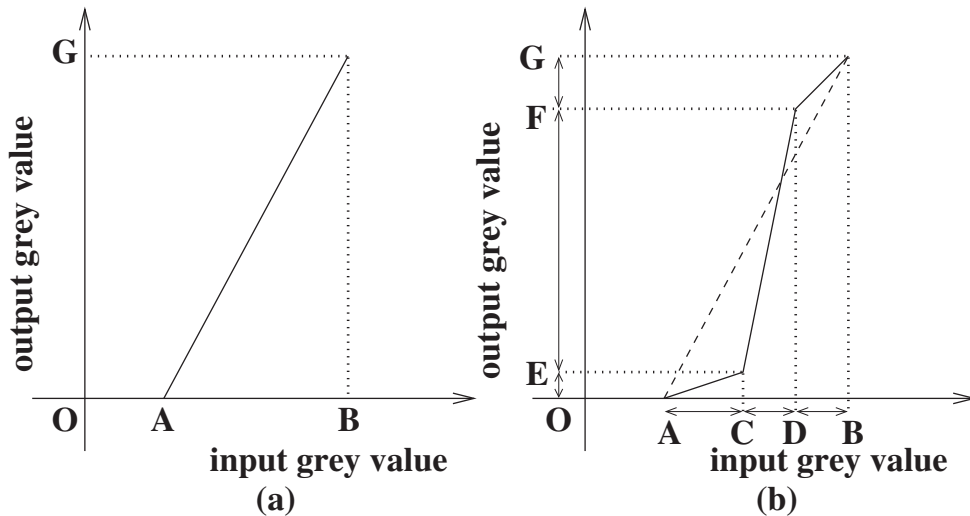


Figure 4.42: (a) A simple stretching (see equation (4.133), on page 367) takes the input range of grey values $[A, B]$ and maps it linearly to the output range $[O, G]$, where $G - O > B - A$. (b) An algorithm that “knows” that grey values in the range $[A, C]$ belong to more or less uniform regions, may suppress the stretching of these values and map them to range $[O, E]$, such that $E - O < C - A$. The same algorithm, “knowing” that values in the range $[C, D]$ often appear in regions of true brightness transitions, may map the grey values in the range $[C, D]$ to the range $[E, F]$, so that $F - E > D - C$. Grey values in the range $[D, B]$ may neither be stretched nor suppressed.

How can we enhance an image by stretching only the grey values that appear in genuine brightness transitions?

Let us consider the image of figure 4.43a. It is a 3-bit 4×4 image. First, let us count how many pairs of grey values of certain type we find next to each other assuming 8-connectivity. We are not interested in ordered pairs, ie $(3, 5)$ is counted the same as $(5, 3)$. The 2D histogram of pairs of values we construct that way occupies only half of the 2D array, as shown at the top of figure 4.43c.

We may select a threshold and say, for example, that pixels that are next to each other and differ by less than 2 grey levels, owe their difference to noise only, and so we do not wish

to stretch, but rather suppress their differences. These pixels correspond to range $[A, C]$ of figure 4.42b. They are the pairs that form the main diagonal of the 2D histogram and the diagonal adjacent to it, as the members of those pairs differ from each other either by 0 or by 1 grey level. The differences of all other pairs are to be stretched. Differences that are to be suppressed should be associated with some negative “force”, that will have to bring the dashed line in figure 4.42b down, while differences that are to be stretched are to be associated with some positive “force”, that will have to bring the dashed line in figure 4.42b up. The more neighbouring pairs a particular grey level participates to, the more likely should be to stretch the mapping curve for its value upwards. So, in the 2D histogram of pairs of values we constructed, we sum the values in each column, ignoring the values along the diagonal strip that represent pairs of values that are to be suppressed. Those are summed separately to form the forces that will have to pull the mapping curve down. The two strings of numbers created that way are shown at the bottom of figure 4.43c as positive and negative forces that try to push the mapping curve upwards or downwards, respectively.

Of course, the mapping function cannot be pushed down and up simultaneously at the same point, so, the two forces have somehow to be combined. We may multiply the negative forces with a constant, say $\alpha = 0.2$, and add the result to the positive forces to form the combined net forces shown in 4.43c. The mapping curve should be stretched so that neighbouring grey values differ proportionally to these numbers. At this point we do not worry about scaling the values to be in the right range. We shall do that at the end.

Next, we work out the cumulative of this string of numbers so that as the grey levels advance, each new grey value differs from its previous one, by as much as the net force we computed for that value. Now, these numbers should be added to the ordinary stretching numbers, ie those represented by the dashed line in 4.42b, which were computed using equation (4.133) of page 367. The ordinary stretching and the calculated cumulative force are added to form the mapping line. The values of this line far exceed the allowed range of 3-bits. So, we scale and round the numbers to be integers between 0 and 7. This is the final mapping curve. The original grey values are mapped to the values in the very bottom line of figure 4.43c. The input values and these values form a look-up table for image enhancement. The mapping curve worked out this way is shown in figure 4.43d. Note that this mapping is not one-to-one. In a real application one may experiment with weight α of the negative forces, to avoid many-to-one mappings.

How do we perform pairwise image enhancement in practice?

The algorithm for such an image enhancement is as follows.

Step 0: If the grey values of the image are in the range $[A, B]$, create a 2D array C of size $(B - A + 1) \times (B - A + 1)$ and initialise all its elements to 0. The elements of this array are identified along both directions with indices from A to B .

Step 1: Accumulate the pairs of grey values that appear next to each other in the image, using 8-connectivity, in the bottom triangle of array C . You need only the bottom triangle because you accumulate in the same cell pairs (g_1, g_2) and (g_2, g_1) . To avoid counting a pair twice, start from the top left corner of the image and proceed from top left to bottom right, by considering for each pixel (i, j) only the pairs it forms with pixels $(i + 1, j)$, $(i + 1, j + 1)$, $(i, j + 1)$ and $(i - 1, j + 1)$, as long as these pixels are within the image boundaries.

Step 2: Decide what the threshold difference d should be, below which you will not enhance

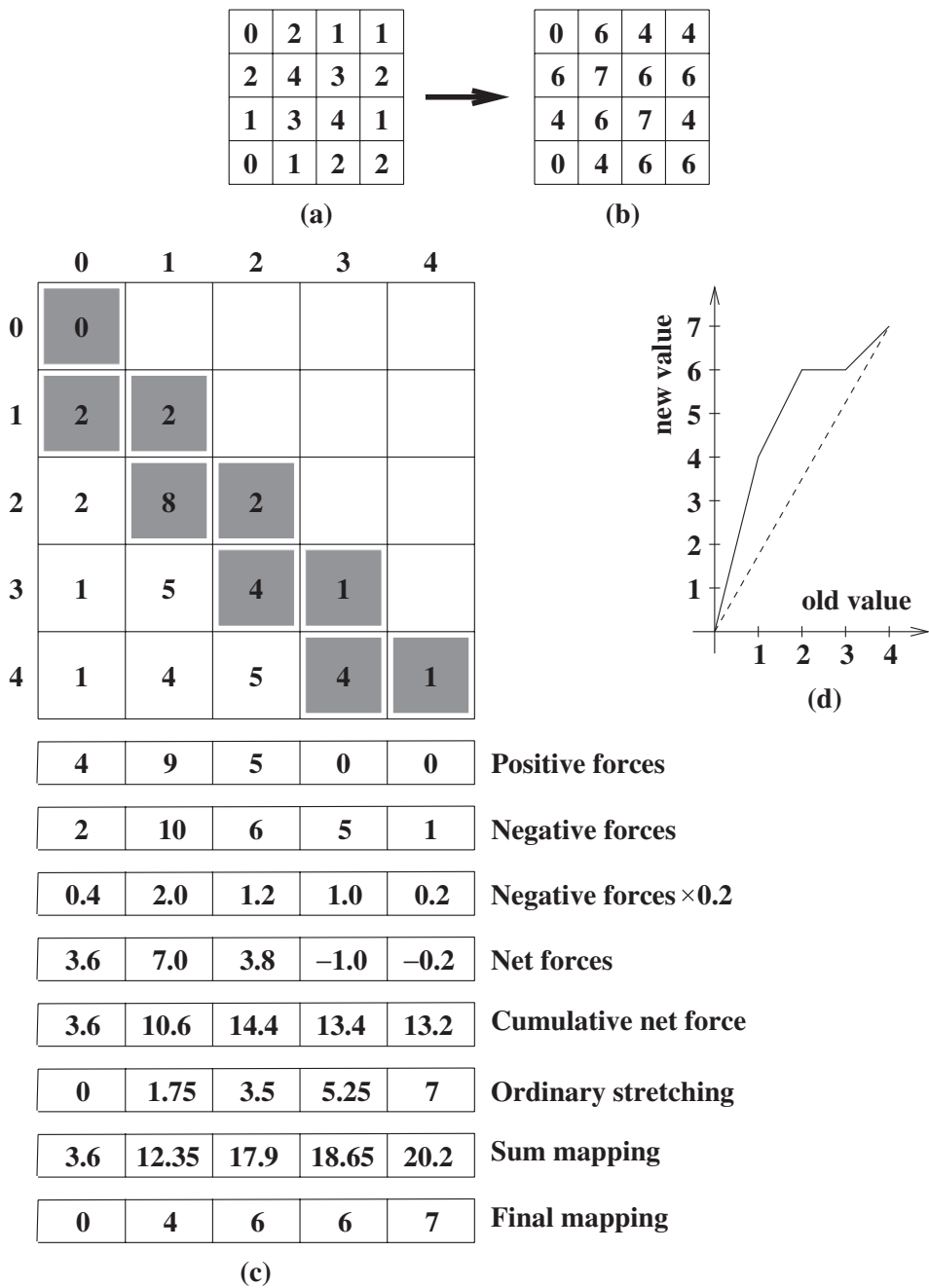


Figure 4.43: An example of image enhancement where pairs of values that appear next to each other and differ by more than a threshold (here equal to 1) are stretched more than other pairs. (a) The original image. (b) The enhanced image. (c) All steps of the algorithm. The grey boxes indicate the cells from which the negative forces are computed. (d) The plot of the final mapping curve.

grey value differences. This determines how wide a strip is along the diagonal of array C , that will be used to form the negative forces.

Step 3: Add the columns of the array you formed in Step 1 that belong to the strip of differences you wish to suppress, to form the string of negative forces:

$$F^-(g) = \begin{cases} C(B, B) & \text{for } g = B \\ C(B-1, B) + C(B-1, B-1) & \text{for } g = B-1 \\ \dots & \\ C(g, g-d+1) + \dots + C(g, g-1) + C(g, g) & \text{for } g > B-d+1 \\ \dots & \\ \sum_{i=g-d}^g C(g, i) & \text{for } g \leq B-d \end{cases} \quad (4.150)$$

Step 4: Add the values of the remaining cells in each column of the 2D array, to form the string of the positive forces:

$$F^+(g) = \begin{cases} 0 & \text{for } g \geq B-d \\ \sum_{i=A}^{g-d-1} C(g, i) & \text{for } g < B-d \end{cases} \quad (4.151)$$

Step 5: Multiply the negative forces with a number α in the range $(0, 1]$ and subtract them point by point from the positive forces, to form the net forces:

$$F^{net}(g) = F^+(g) - \alpha F^-(g) \quad \text{for } A \leq g \leq B \quad (4.152)$$

Step 6: Accumulate the net forces by starting from left to right and creating a running sum, each time adding the next force:

$$S(g) = \sum_{i=A}^g F^{net}(i) \quad \text{for } A \leq g \leq B \quad (4.153)$$

Step 7: Create the mapping from the old to new values, using equation (4.133), $g_{new}(g)$.

Step 8: Add the corresponding values you produced in Steps 6 and 7:

$$\tilde{g}(g) = S(g) + g_{new}(g) \quad \text{for } A \leq g \leq B \quad (4.154)$$

Step 9: Scale and round the resultant values:

$$\tilde{g}_{new}(\tilde{g}(g)) = \left\lfloor \frac{\tilde{g}(g) - \tilde{g}(A)}{\tilde{g}(B) - \tilde{g}(A)} G + 0.5 \right\rfloor \quad (4.155)$$

Step 10: Use the above formula to enhance the image.



(a)



(b)



(c)

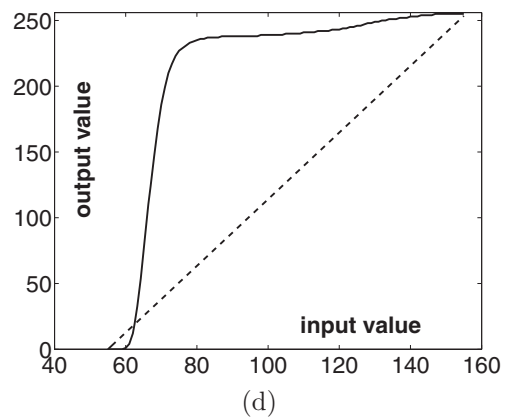


Figure 4.44: “A Catholic Precession” (size 512×425). (b) Enhanced by simple stretching. (c) Enhanced by considering pairs of pixels, using parameters $d = 1$ and $\alpha = 0.5$. (d) The mapping function for stretching (dashed line) and its modification by considering neighbouring pairs of pixels (continuous line).

Note that the above algorithm may be applied globally or locally, inside running windows. When running windows are used, we only change the value of the pixel in the centre of the window and then shift the window by one pixel and repeat the whole process.

Figure 4.44 shows an original image with very low contrast, its enhanced version by simple stretching its range of grey values, and its enhanced version by applying the above algorithm. Figure 4.44d shows the mapping function between the original and the final grey image values. The dashed line is the mapping of simple stretching, while the continuous line is the mapping obtained by the above algorithm. Figure 4.45 shows a bad image and various enhanced versions of it.



(a) Original image



(b) After global histogram equalisation



(c) Local histogram equalisation 81×81



(d) Enhancement with pairwise relations

Figure 4.45: Enhancing the image of “The Hanging Train of Wuppertal”. For the enhancement with the pairwise relations approach, $\alpha = 0.1$ and $d = 3$.

4.5 Generic deblurring algorithms

Proper image deblurring will be discussed in the next chapter, under image restoration. This is because it requires some prior knowledge of the blurring process in order to work correctly. However, one may use some generic methods of deblurring, that may work without any prior knowledge.

If we plot a cross-section of a blurred image, it may look like that of figure 4.46a. The purpose of deblurring is to sharpen the edges, so that they look like those in 4.46b. We shall discuss here some algorithms that may achieve this: mode filtering, mean shift and **toboggan² contrast enhancement**.

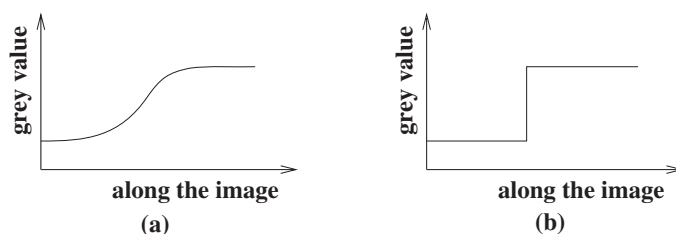


Figure 4.46: A cross section of a blurred image looks like (a). The purpose of deblurring algorithms discussed in this section is to make profiles like (a) become like (b).

How does mode filtering help deblur an image?

It has been shown that repeated application of the mode filter (see page 333) may result in an image made up from patches of uniform grey value with sharp boundaries. The mode may be applied with or without the use of weights. Figure 4.47 shows an image blurred due to shaken camera, and the results of the successive application of mode filtering. The algorithm took 90 iterations to converge. The weights used were

$$\begin{pmatrix} 1 & 3 & 1 \\ 3 & 5 & 3 \\ 1 & 3 & 1 \end{pmatrix} \quad (4.156)$$

These weights were chosen so that the chance of multiple modes was reduced, and the central pixel was given reasonable chance to survive, so that image details might be preserved. What the algorithm does in the case of multiple modes is very critical for the outcome. For these results, when multiple modes were observed, the algorithm worked out the average mode and rounded it to the nearest integer before assigning it to the central pixel. Note that by doing that, we create grey values that might not have been present in the original image. This leads to slow convergence, and ultimately to the creation of artifacts as one can see from figures 4.47e and 4.47f. Figure 4.48a shows the result of applying the mode filtering with the same weights, but leaving the value of the pixel unchanged if multiple modes occurred.

²Toboggan is a type of sledge (originally used by the Canadian Indians) for transportation over snow.